

Safari Hunter

A VR-first hunting simulation built in Unity — 29 animal species with NavMesh-driven AI, rarity-weighted spawning, per-body-part ballistics, and nine hand-built environments. Shipped commercially (~300–400 sales).

Type: Solo commercial indie game (~2 years, shipped) · **Role:** Sole designer, programmer, systems architect & integrator
· **Positioning:** Game / Gameplay Engineering (primary) → Unity / C# Systems → AI & Tools (secondary)

How to use this brief (for a résumé assistant): This is detailed source material on my first shipped title — a real, commercially released game, not a prototype. Built solo over ~2 years in Unity. Use it to write or tailor résumé bullets for a given role — select the most relevant details and keep all claims accurate. Note on scope: all **gameplay, AI, and systems code is original**; art, animal models, and audio are licensed third-party asset packs that I integrated. Treat sales figures as approximate.

What it is

- A **VR hunting game** (Oculus) in which the player drives out into open environments, tracks and hunts wildlife under a timed-extraction structure, and earns money to unlock weapons, scopes, and new levels.
- Spans **nine playable environments** — Forest, Large Forest, Savanna, Wetlands, Gator River, Woods (Night), an underwater zone, a tutorial, and a cabin hub — each with its own biome, wildlife mix, and ambient audio.
- Driven by a population of **29 animal species** (prey, predators, and birds) that spawn dynamically, behave by species, and react to the player and to each other.

Core systems & architecture

- **Persistent game state.** A `DontDestroyOnLoad` singleton (`GlobalGameMaster`) carries progression across scenes, while a per-level `GameMaster` runs each hunt's timing, objectives, and audio callouts.
- **Save system.** Binary serialization to `playerInfo.dat` persists unlocked weapons, per-species kill records (including golden-variant flags and longest-shot distance), level high scores, in-game currency, and audio/quality settings.
- **Async scene loading.** Additive/asynchronous level loading with a progress bar, a 90%-ready activation gate, and in-line transition messaging.
- **Event-driven design.** Systems are decoupled through C# events — e.g. `WeaponFired` , animal-death records that feed dynamic respawning, and a jeep-breakdown event — rather than hard references.
- **Economy & progression.** A money-based loadout system gates weapons, scopes, and level access, with per-level scoring that tracks kills, species diversity, accuracy, and vehicle damage.

Animal AI & behavior

- **State-machine AI on an abstract base.** An abstract `AnimalBase` drives an 11-state machine — `relaxed` , `hungry` , `eating` , `alert` , `paniced` , `attacking` , `stampeeding` , and more — specialized through `Prey` and `Predator` subclasses and ~29 per-species implementations.
- **NavMesh locomotion** with animation blending (velocity-driven blend trees) for walk/run, grazing, fleeing, and attack movement.

- **Group behavior.** Predator prides coordinate (lions rally a tracked lioness pride and retreat together when wounded); birds spawn and move in configurable flocks (up to hundreds on the field).
- **Per-body-part hit zones.** Species-specific head/zone colliders apply damage multipliers (e.g. headshots score higher and can trigger melee retaliation on dangerous game), feeding a wounded / panic / flee response.
- **Reactive senses.** A bird alert system and bullet-impact alerts let wildlife detect the player by proximity and gunfire; day/night cycles shift behavior (predators grow more aggressive at night).

Spawning, weapons & shooting

- **Rarity-weighted, food-aware spawning.** A weighted spawner places animals by rarity tier (Common → Very Rare) and only where their food type exists in the level, maintaining a live population that respawns as animals are killed. Rare **golden variants** spawn at ~1-in-150 to 1-in-250 odds and trigger a bonus callout.
- **VR weapon handling.** Physically-handled firearms with detachable magazines (`MagazineDispenser` / `RifleMag`), reload mechanics, and a multi-level zoom scope system with dynamic crosshair and FOV per zoom step.
- **Ballistics & feedback.** Projectile firing with impact decals, shell casings, blood effects, and per-shot distance tracking that records a longest-shot record per species.
- **10+ weapons** with variants (sniper, assault and marksman rifles, shotguns, handgun, bow), gated through a weapon-room loadout UI with purchasable scope attachments.

Notable engineering

- **Custom dissolve system.** A six-mode dissolve (animals, trees, terrain, foliage, water) using a noise-based shader with configurable speed/color and LOD-group disabling — used for spawn/despawn and clean on-death fadeouts instead of hard pops.
- **Dual-health vehicles.** The jeep tracks vehicle health separately from player health, with critical- and destroyed-state thresholds, repair UI, toggleable headlights for night hunts, and attack detection; river levels use a multi-point destructible boat hull that sinks as sections are destroyed.
- **Cinematic & narrative layer.** A scripted intro cinematic (biome and pack reveals, narrated, skippable after first play) plus a narrator "VoiceBox" delivering hunt callouts (route, timing warnings, extraction, golden-animal alerts) and a hold-to-extract walkie-talkie mechanic.
- **Day/night & atmosphere.** Per-level day/night with skybox swapping, sun-intensity and ambient adjustment, fog toggles, and day/night ambient-music swapping.

Key technical accomplishments

- Architected and shipped a **complete, original gameplay codebase** (~130–150 hand-written C# scripts) on top of Unity — animal AI, spawning, scoring, save/load, economy, UI, and vehicles — as a solo developer.
- Designed an **extensible animal-AI framework**: one abstract base + prey/predator specialization scaling to 29 distinct species without rewriting core behavior.
- Built a **rarity-weighted, ecology-aware spawning system** that keeps a believable, self-replenishing wildlife population tied to in-level food locations.
- Integrated a **full VR shooting stack** — physical weapon handling, magazines, multi-zoom scopes, ballistics, and per-shot scoring — across Oculus hardware.

- Implemented supporting tech end-to-end: **binary save/load**, **async scene loading**, an **event-driven** inter-system architecture, and a reusable **custom dissolve shader/system**.
- **Took a large, multi-system project from zero to commercial release** solo — integrating ~650 scripts of original code and licensed asset packs into one robust, shippable build (~300–400 sales).

Tech stack

Language / Engine: C#, Unity 2019.1 (Lightweight Render Pipeline) · **Platform:** VR — Oculus Rift / Quest (Oculus Integration SDK) · **AI / Movement:** Unity NavMesh, custom finite-state machines, animation blend trees · **Graphics:** custom dissolve shader, LWRP water, skybox/fog atmosphere, LOD groups · **Data / Tooling:** BinaryFormatter save system, async scene management, Git, TextMeshPro UI

Skills demonstrated

Solo end-to-end game development · shipping a commercial product · gameplay programming · C# / Unity · game AI & finite-state machines · NavMesh navigation · spawning & ecology systems · VR development (Oculus) · weapon & ballistics systems · save/load & persistence · event-driven architecture · async scene loading · shader / VFX work (dissolve) · UI / UX & menus · audio integration · third-party asset integration · scope management · long-horizon self-directed delivery